CEUB

EDUCAÇÃO SUPERIOR

Desenvolvimento de Sistemas

ceub.br

CEUB
EDUCAÇÃO SUPERIOR

Aula 2 - Fundamentos do Backend e POO

Agenda

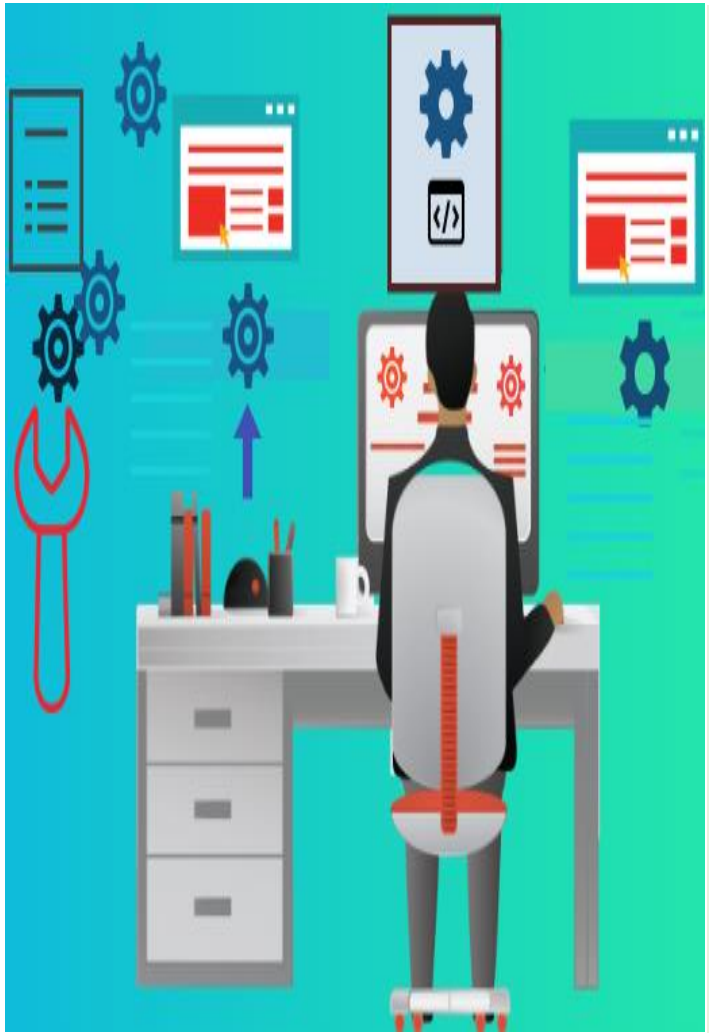
- Desenvolvimento Back-End
- Arquitetura de aplicações Back-End
- Considerações no desenvolvimento Back-end
- Principais tecnologias utilizadas em back-end
- Fatores que influenciam na escolha de uma linguagem de Programação
- Revisão de Programação Orientada a Objetos.

Desenvolvimento Back-End

O desenvolvimento back-end é a parte da aplicação que você não vê diretamente.

- **É a coração da aplicação, que dá vida a tudo.**
- **É o guardião das Regras de Negócio, Segurança e Integridade de Dados.**
- **Backend Moderno = APIs REST/gRPC + Cloud Native + Escalabilidade.**

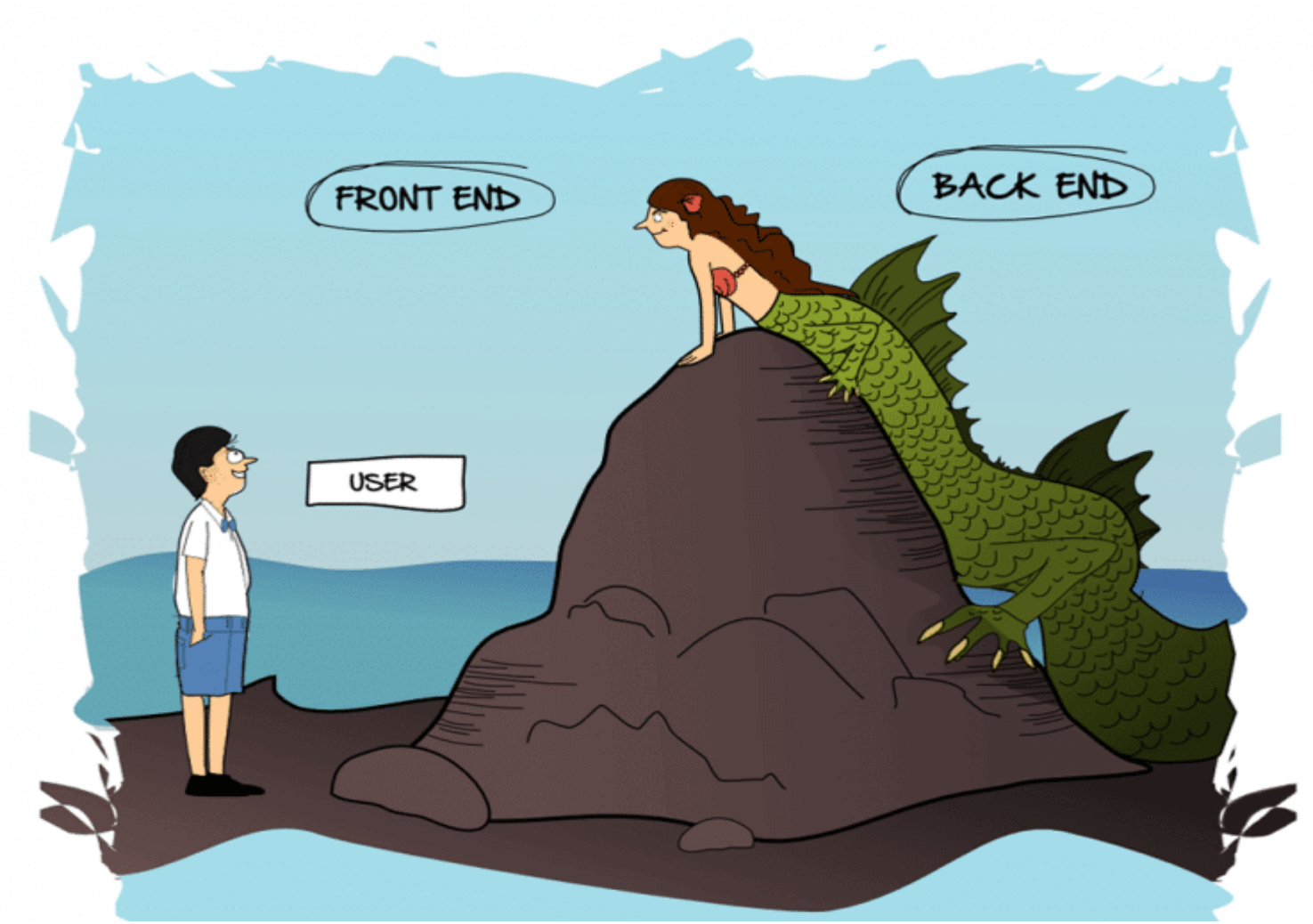
Desenvolvimento Back-End



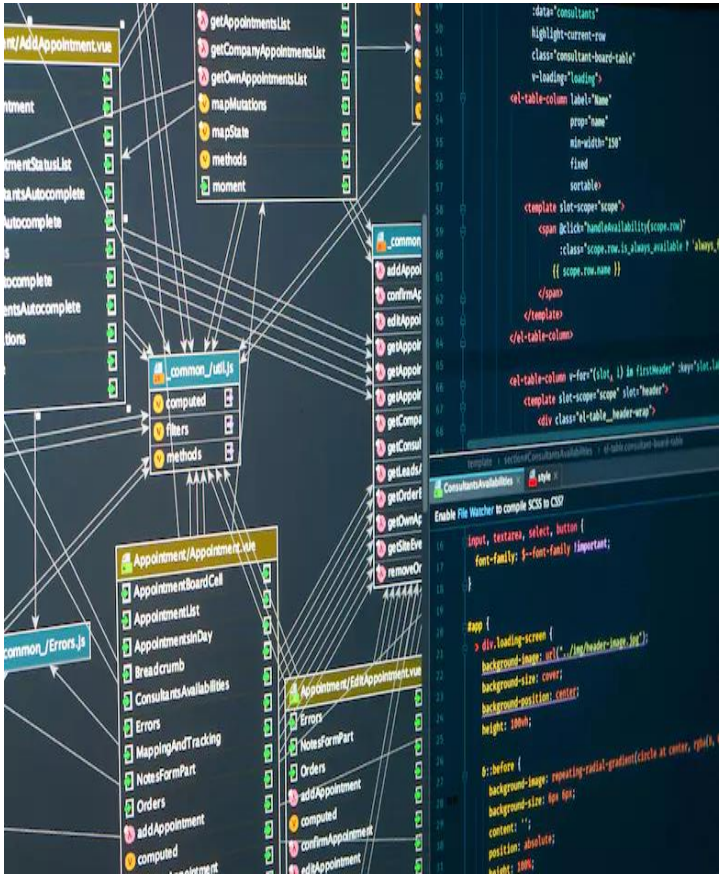
O Desenvolvimento back-end é responsável por lidar com a lógica de negócio, o armazenamento e a segurança dos dados, a integração com outras APIs e serviços, e a comunicação com o front-end.

O desenvolvimento back-end envolve o uso de linguagens de programação, frameworks, bancos de dados, servidores web, ferramentas de teste e deploy.

Desenvolvimento Back-End



Arquitetura de aplicações Back-End



A arquitetura de uma aplicação back-end é a estrutura que define como os componentes da aplicação se organizam e interagem entre si.

Ela é fundamental para garantir a escalabilidade, confiabilidade e segurança da aplicação.

Arquitetura de aplicações Back-End

A arquitetura de uma aplicação back-end pode variar de acordo com as necessidades e os requisitos do projeto, mas geralmente segue alguns princípios e padrões comuns, tais como:

- **Monolítica**
- **Padrões MVC**
- **Arquitetura N-Layered**
- **Domain-Driven Design (DDD)**
- **Arquitetura Hexagonal**
- **Arquitetura Onion**
- **Clean Architecture**
- **Microsserviços**
- **Serverless**

Considerações no desenvolvimento Back-end

Padrões de Projeto:

- São soluções pré-definidas para problemas comuns de arquitetura.
- O uso de padrões de design pode ajudar a melhorar a qualidade e a confiabilidade da aplicação.

Segurança:

- A segurança deve ser uma consideração fundamental na arquitetura da aplicação.
- É importante implementar medidas de segurança para proteger a aplicação contra ataques e acessos não autorizados.

Considerações no desenvolvimento Back-end

Para lidar com a crescente complexidade, a arquitetura back-end evoluiu, oferecendo diversos padrões:

- **N-camadas: divisão em camadas com funções específicas.**
- **DDD: foco no domínio de negócio para maior flexibilidade.**
- **Hexagonal/Onion/Clean Architecture : desacoplamento da lógica de negócio da interface e infraestrutura.**

Considerações no desenvolvimento Back-end

Compreender esses estilos permite:

- **Tomar decisões informadas.**
- **Construir sistemas robustos, escaláveis e manuteníveis.**
- **Alinhar o sistema com o domínio de negócios.**
- **Garantir crescimento e adaptabilidade futuros.**
- **Dominar a arquitetura back-end é essencial para o desenvolvimento de software moderno.**

Considerações no desenvolvimento Back-end

ATENÇÃO

A escolha da arquitetura ideal para uma aplicação back-end depende de diversos fatores, como o tamanho e a complexidade da aplicação, os requisitos de escalabilidade e performance, e o orçamento disponível.

O desenvolvimento back-end é uma área em constante crescimento, com diversas oportunidades para profissionais qualificados.

Principais tecnologias utilizadas em back-end

No desenvolvimento Back-End são utilizadas diversas tecnologias, abaixo temos alguns exemplos:

- **Linguagens de programação: Java, C#, Python, JavaScript (Node.js), PHP, Ruby, Go, entre outras.**
- **Frameworks: Django, Spring Boot, .Net Core Laravel, Express, Rails, etc.**
- **Bancos de dados: Oracle, SQL Server, MariaDB, PostgreSQL, MongoDB, Cassandra, Redis, Neo4J, etc.**
- **Servidores: Apache, IIS, Nginx, Tomcat, etc.**

O que é uma Linguagem de Programação?

- **Na programação de computadores, uma linguagem de programação serve como meio de comunicação entre o indivíduo que deseja resolver um determinado problema e o computador.**

- **A linguagem de programação deve fazer a ligação entre o pensamento humano (muitas vezes de natureza não estruturada) e a precisão requerida para o processamento pelo computador.**

O que é uma Linguagem de Programação?

Uma linguagem de programação auxilia o programador no processo de desenvolvimento de software:

- **Análise de Requisitos;**
- **Projeto;**
- **Implementação;**
- **Teste;**
- **Verificação;**
- **Manutenção do software;**

O que é uma Linguagem de Programação?

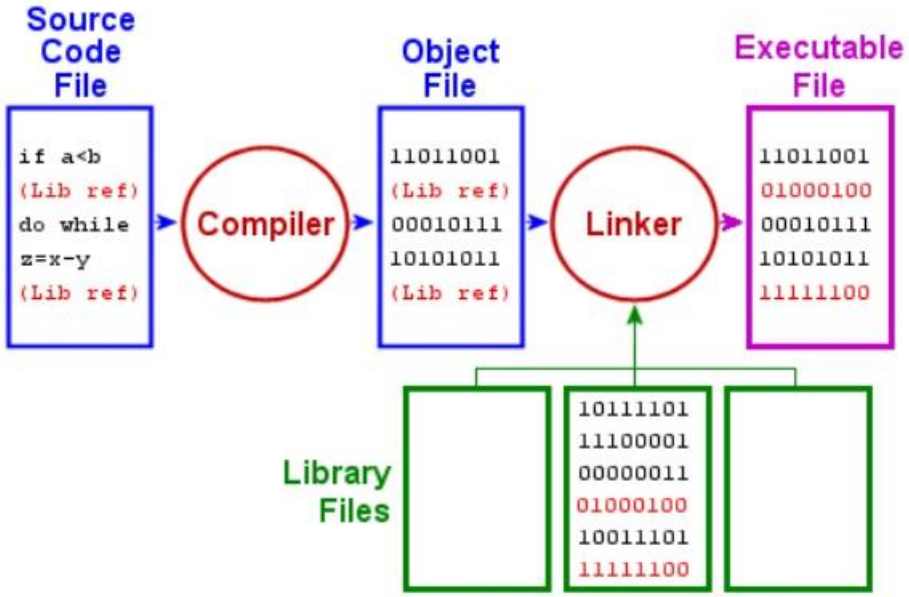
Uma linguagem de programação é uma linguagem destinada para ser usada por uma pessoa para expressar um processo através do qual um computador possa resolver um problema.

Os modelos/paradigmas de linguagens de programação correspondem a diferentes pontos de vista dos quais processos podem ser expressados.

Exemplos: Imperativo, orientado a objeto, funcional, lógico

O que é uma Linguagem de Programação?

É uma formar de interagir com as máquinas(computadores), tornando operacionais os programas escritos em linguagens de alto nível que necessitam ser traduzidos para linguagem de máquina.



O que é uma Linguagem de Programação?

A conversão de um código em linguagem alto nível para linguagem de máquina é realizada através de sistemas especializados:

Compiladores ou Interpretadores

Esses sistemas recebem como entrada uma representação textual da solução de um problema (expresso em uma linguagem fonte) e produzem uma representação do mesmo algoritmo expresso em uma linguagem de máquina.

CEUB
EDUCAÇÃO SUPERIOR

Por que estudar Linguagens de Programação?

Ampliar e melhorar a capacidade de resolver problemas e expressar ideias

- **Conhecimento amplo dos recursos de linguagem reduz as limitações no desenvolvimento de software;**
- **A melhor compreensão das funções e implementação das estruturas de uma linguagem de programação nos leva a usar a linguagem de modo a extrair o máximo de sua funcionalidade e eficiência;**
- **Recursos ou facilidades podem ser simulados.**

Por que estudar Linguagens de Programação?

Ampliar o leque de conhecimento sobre linguagens de programação:

- **Algumas linguagens são mais apropriadas para resolver determinados problemas;**
- **Torna mais fácil aprender novas LPs e acompanhar a tecnologia(quanto mais LPs você souber, mais fácil se torna aprender uma nova LP).**
- **Escolher a melhor linguagem para um problema específico devido ao conhecimento de novos recursos é difícil para:**
  - ❖ **Programadores antigos;**
  - ❖ **Desenvolvedores sem educação formal;**

CEUB
EDUCAÇÃO SUPERIOR

Por que estudar Linguagens de Programação?

Entender a importância da implementação no processo de desenvolvimento de software:

- **Leva a um entendimento do porquê das linguagens serem projetadas de determinada maneira;**
- **Possibilitar o entendimento sobre quais linguagens de programação podem ser utilizadas e o resultado direto dessas escolhas**
- **Possibilita o desenvolvimento de códigos, programas mais efetivos (eficientes e eficazes), aumentando o seu vocabulário de construção úteis de programação**

Por que estudar Linguagens de Programação?

Maior capacidade para aprender novas linguagens:

- **Na computação, é fundamental que haja um aprendizado contínuo e eficiente de linguagens de programação**
- **Compreender os conceitos gerais das linguagens torna mais fácil entender como eles são incorporados na linguagem que está sendo aprendida;**

- **TIOBE Index for Janeiro 2026:**

- https://www.tiobe.com/tiobe-index/

Desenvolvimento de Sistemas - DS

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	^	C	10.99%	+2.13%
3	3		Java	8.71%	-1.44%
4	2	v	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	^	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	^	Delphi/Object Pascal	1.98%	+0.19%
10	18	^^	R	1.82%	+0.81%
11	32	^^	Perl	1.63%	+1.14%
12	10	v	Fortran	1.61%	-0.42%
13	14	^	Rust	1.51%	+0.34%
14	15	^	MATLAB	1.40%	+0.34%
15	13	v	PHP	1.38%	-0.00%
16	7	vv	Go	1.24%	-1.37%
17	12	vv	Scratch	1.24%	-0.31%
18	26	^^	Ada	1.19%	+0.54%
19	17	v	Assembly language	1.07%	+0.05%
20	25	^^	Kotlin	0.97%	+0.23%

Por que estudar Linguagens de Programação?

Avanço global da computação:

Aprender programação é uma das habilidades do século 21 e deveria ser tão importante quanto ler ou escrever.

Ter a capacidade de entender que nem sempre as melhores linguagens de programação são as mais populares.

Desenvolver o entendimento que existem muitas linguagens de programação, cada uma com a finalidade específica para resolução problemas.

CEUB
EDUCAÇÃO SUPERIOR

Fatores que influenciam na escolha de uma linguagem

- **Implementação**
  - **Disponibilidade quanto à plataforma**
  - **Eficiência: velocidade de execução do programa objeto**

- **Competência na Linguagem de Programação**
  - **Experiência do Programador**
  - **Competência do grupo envolvido**

- **Portabilidade**
  - **Necessidade de executar em diferentes plataformas**

CEUB
EDUCAÇÃO SUPERIOR

Fatores que influenciam na escolha de uma linguagem

- **Ambientes de Programação**
  - **Ferramentas para desenvolvimento de software diminuem o esforço de programação**
  - **Bibliotecas**

- **Modelo de Computação**
  - **Aplicação vs modelo de computação**
  - **Ex.: para realização de busca heurística é adequado o Paradigma Lógico, para simulações, o Paradigma Orientado a Objeto**

Paradigmas de Programação

Quando uma linguagem de programação é criada, a partir das suas características, ela é categorizada em um ou mais paradigmas.

A definição do dicionário Aurélio para "paradigma":

- **Algo que serve de exemplo geral ou de modelo.**
- **Conjunto das formas que servem de modelo de derivação ou de flexão.**
- **Conjunto dos termos ou elementos que podem ocorrer na mesma posição ou contexto de uma estrutura.**

Paradigmas de Programação

O paradigma de uma linguagem de programação é a sua identidade. Corresponde a um conjunto de características que, juntas, definem como ela opera e resolve os problemas. Algumas linguagens, inclusive, possuem mais de um paradigma, são as chamadas multi paradigmas, Ex JScript.

Alguns dos principais paradigmas utilizados hoje no mercado:

- **Funcional**
- **Lógico**
- **Declarativo**
- **Imperativo**
- **Orientado a objetos**
- **Orientado a eventos**

Paradigmas de Programação

Paradigma Orientado a Objetos

Esse é, entre todos, talvez o mais difundido. Nesse paradigma, ao invés de construirmos nossos sistemas com um conjunto estrito de procedimentos, assim como se faz em linguagens "fortemente imperativas" como o Cobol, Pascal etc.

Na orientação a objetos utilizamos uma lógica bem próxima do mundo real, lidando com objetos, estruturas que já conhecemos e sobre as quais possuímos uma grande compreensão.

O paradigma orientado a objetos tem uma grande preocupação em esconder o que não é importante e em realçar o que é importante.

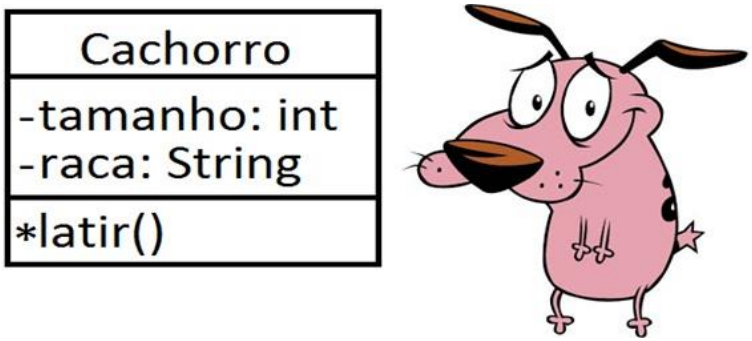
Paradigmas de Programação

Paradigma Orientado a Objetos

Nele, implementa-se um conjunto de classes que definem objetos.

Cada classe determina o comportamento (definido nos métodos) e estados possíveis (atributos) de seus objetos, assim como o relacionamento entre eles.

Esse é o paradigma mais utilizado em aplicações comerciais e as principais linguagens o implementam: C#, C#, PHP, Ruby, C++, Python etc.



Introdução a Programação Orientada a Objetos

Conceitos

"Uma nova maneira de pensar os problemas utilizando conceitos do Mundo Real. O componente fundamental é o objeto que combina estrutura e comportamento em uma única entidade" Raumbaugh

"Sistema orientado a objetos é uma coleção de objetos que interagem entre si" Bertrand Meyer

CEUB
EDUCAÇÃO SUPERIOR

O que é a Orientação a Objetos?

Orientação a Objetos (OO) é um paradigma de programação;

- **Forma diferente de pensar e construir algoritmos e programas para computadores;**

Conceito introduzido originalmente na linguagem de programação Simula 67:

- **Ole-Johan Dahl e Kristen Nygaard;**
- **Oslo, Noruega em 1967;**
- **Nos anos 70 surge Smalltalk, a primeira linguagem totalmente em Orientação a Objeto (O.O)**

  C++, evolução de C, já possuía conceitos O.O

Na década de 80 praticamente todas as linguagens já usavam conceitos O.O

- **Delphi**
- **JAVA**
- **C#(2002)**

O que é a Orientação a Objetos?

A Programação Orientada a Objetos (POO) é praticada com novos conceitos:

- **Modelagem de Objetos;**
- **Atributos e comportamentos;**

Tudo passa a ser assumido como objetos, tal como acontece no mundo real;

- **Isso trás diferenças substanciais quanto à forma de se pensar na construção de algoritmos!**

Os 4 pilares da Programação Orientada a Objetos

Para que uma linguagem possa ser enquadrada no paradigma de orientação a objetos, ela deve atender a quatro tópicos bastante importantes:

- **Abstração**
- **Encapsulamento**
- **Herança**
- **Polimorfismo**

CEUB
EDUCAÇÃO SUPERIOR

Os 4 pilares da Programação Orientada a Objetos

Abstração

A abstração consiste em um dos pontos mais importantes dentro de qualquer linguagem Orientada a Objetos.

Como estamos lidando com uma representação de um objeto real (o que dá nome ao paradigma), temos que imaginar o que esse objeto irá realizar dentro de nosso sistema.

Habilidade de se concentrar nos aspectos essenciais do sistema, ou um contexto qualquer, ignorando o que é supérfluo.

Programação Orientada a Objetos

Elementos básicos

Objetos

Classes

Instâncias

Classe

- A Classe é o molde, a planta, o esquema, o modelo a ser seguido pelos objetos
- A planta da casa é o modelo que as casas construídas terão
- Porém não é possível morar na planta da casa, apenas na casa já construída
- A Classe define as características da casa e as funções que ela terá: parte elétrica, hidráulica, saneamento e etc.

CEUB
EDUCAÇÃO SUPERIOR

CLASSES

Sintaxe básica de criação de classes em C#

Nomes de classes não podem ser exatamente iguais às palavras reservadas de C# e ou Java.

- **Caracteres maiúsculos e minúsculos são diferenciados em C#: as palavras Class, CLASS, ClAsS e class são consideradas como sendo diferentes, e somente a última pode ser usada para declarar uma classe.**

Ex em C# e Java

```
public class NomeClasse {

}
```

```
class NomeClasse {

}
```

Ex em Python

```
class NomeClasse:
```

Instancia

Instancia é sinonimo de um objeto;

O ato de instanciar uma classe é o ato de criar um objeto.

Em C#, uma instância de uma classe é um novo objeto criado dessa classe, com o operador new.

Instanciar uma classe é criar um novo objeto do mesmo tipo dessa classe.

Uma classe somente poderá ser utilizada após ser instanciada.

Objeto

- Objetos são utilizados para representar conceitos do mundo real
- Objetos seguem fielmente as especificações de suas Classes
- Os Objetos são instâncias concretas das Classes
- As casas são instâncias concretas das plantas que lhes deram origem

Criando um Objeto

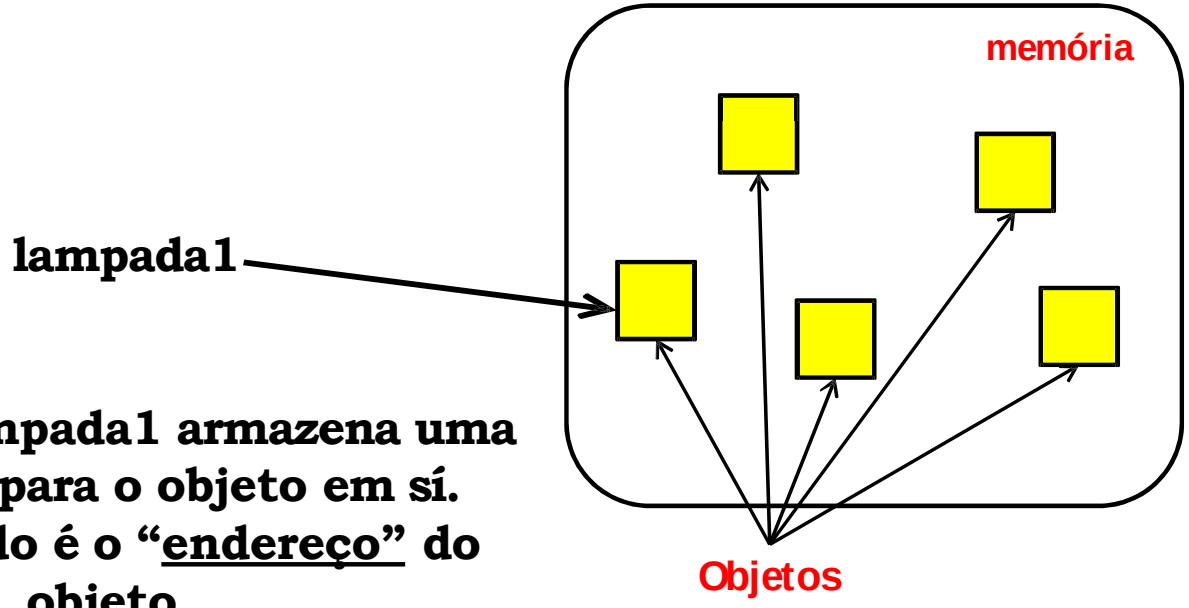
- Operador *new* cria um novo objeto a partir de uma classe especificada (cria uma instância)
- Retorna uma referência para esse objeto

```
new <tipo_classe> ([parametro, parametro, ...]);
```

[ ] = Opcionais
< > = Identificadores e palavras reservadas

Criando um Objeto

Lampada lampada1 = new Lampada();



A variável lampada1 armazena uma referência para o objeto em sí. Seu conteúdo é o "endereço" do objeto

CEUB
EDUCAÇÃO SUPERIOR

CLASSES

Sintaxe básica de criação de objetos em C#

Para criar (construir, instanciar) uma classe, basta usar a palavra chave new.

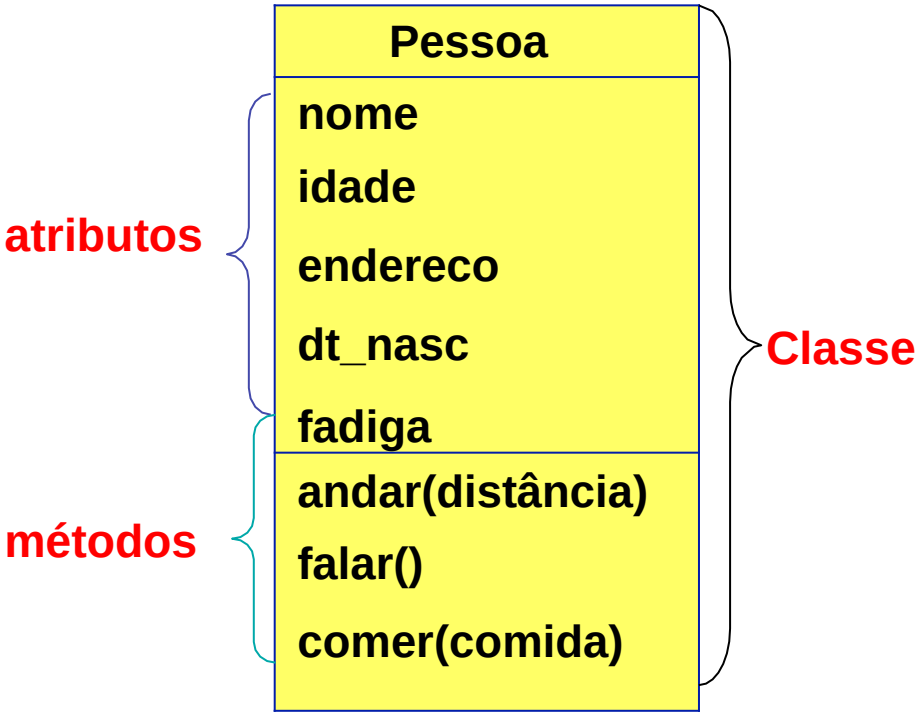
Ex

```
class NomeClassePrincipal {
 public static void main(String[] args) {
      NomeClasse meuObjeto;
      meuObjeto = new NomeClasse();

      ou

      NomeClasse meuObjeto = new NomeClasse();

  }
}
```

Atributos

- São as variáveis de instância
  - Fazem parte de cada objeto (instância)
- Declarada fora dos métodos
- "Vivem" enquanto o objeto "viver“
- Obs: Todo objeto possui um identificador chamado this, que é uma referência para o próprio objeto.

UML



- A classe é composta por atributos (propriedades) e métodos (ações / mensagens).
- Os atributos possuem nomes significativos que melhor definem o dado de negócio que representam.
- Os métodos são sempre ações que operam sobre os atributos da classe.
- Em uma boa programação OO, somente os métodos de objeto podem alterar o estado do objeto (atributos).
- Os métodos recebem (ou não) parâmetros que permitem customizar uma ação sobre os atributos.

Classe Completa

```
public class Lampada
{
public boolean estadoLampada;

        public Lampada(){
                estadoLampada = false;
        }

        public void acenderLampada() {
                estadoLampada = true;
        }

        public void apagarLampada() {
                estadoLampada = false;
        }

        public boolean verEstadoLampada() {
                return estadoLampada;
        }
}
```

Atributos

Construtor

Métodos

Método

- É possível que uma Classe possua 2 métodos com o mesmo nome?
- Sim, é possível, mas devem ter parâmetros diferentes (quantidade e tipo)!!!
  O nome que se dá a isso é sobrecarga ou clonagem!

Exemplo:

```
double calcularMedia(double nota1, double nota2){
        return (nota1+nota2)/2;
}

double calcularMedia(double nota1, double nota2, int peso1, int peso2){
        return (nota1*peso1+nota2*peso2)/peso1+peso2;
}
```

Construtor

- Uma classe pode conter vários construtores
  - Mesmo nome da classe
  - Não possui retorno
  - Diferença na quantidade e tipo dos parâmetros

Sobrecarga

- Construtor padrão é fornecido
  - Se não houver pelo menos um definido
  - Não possui parâmetros

- É chamado na execução do new

Exemplo OO C# Criação de Classe Funcionário

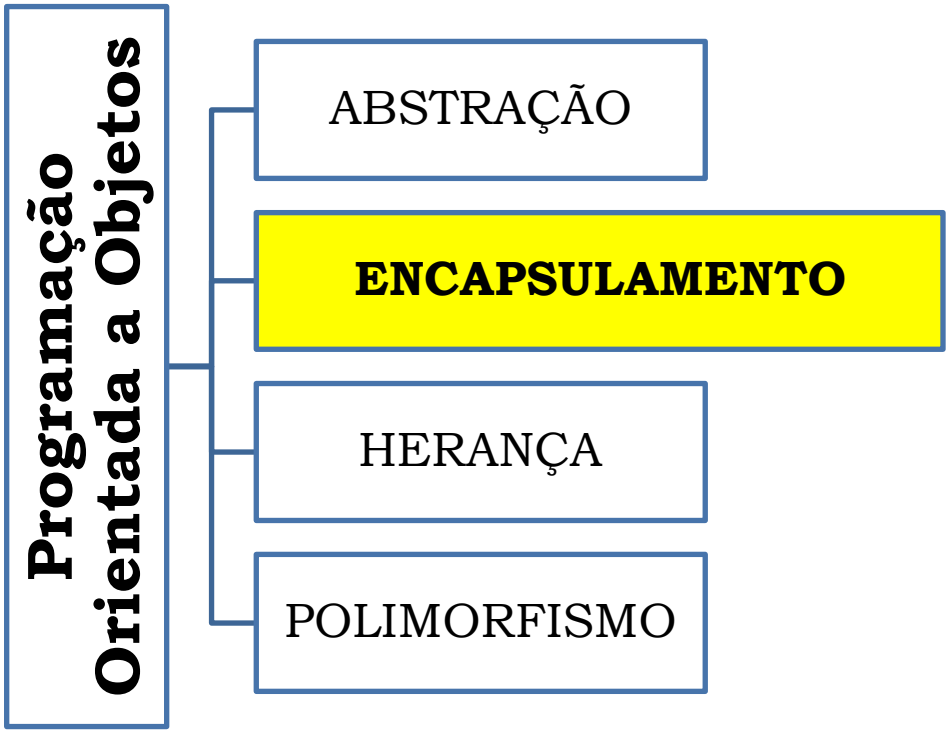
```
using System;
using System.Collections.Generic;
using System.Text;

namespace ExemploOO.BLL
{
    class Funcionario
    {
        private string _nome;
        private int _idade;
        private string _sexo;
        private double _salario;

        public string Nome { get => _nome; set => _nome = value; }
        public int Idade { get => _idade; set => _idade = value; }
        public string Sexo { get => _sexo; set => _sexo = value; }
        public double Salario { get => _salario; set => _salario = value; }

        //MÉTODOS
        public double calculaDecimoTerceiro(int mesesTrabalhos)
        {
            double decimoTerceiro;
            decimoTerceiro = _salario * mesesTrabalhos / 12;
            return decimoTerceiro;
        }
        public void calculaFerias()
        {
            double ferias;
            ferias = _salario + _salario / 3;
            Console.WriteLine("O SALARIO DE FERIAS É R$: " + ferias + " Reais");
        }

    }
}
```

Exemplo OO C# Aplicação Console

```
using ExemploOO.BLL;
using System;

namespace ExemploOO
{
    class Program
    {
        static void Main(string[] args)
        {
            Funcionario funcionario = new Funcionario();

            Console.WriteLine("Digite o nome do funcionário");
            funcionario.Nome = Console.ReadLine();
            Console.WriteLine("Digite a idade do funcionário");
            funcionario.Idade = int.Parse(Console.ReadLine());
            Console.WriteLine("Digite o sexo do funcionário");
            funcionario.Sexo = Console.ReadLine();
            Console.WriteLine("Digite o salário do funcionário");
            funcionario.Salario = double.Parse(Console.ReadLine());

            Console.WriteLine("O funcionário {0} possui {1} anos,  ele é do sexo {2} e seu salário é R$ {3}", funcionario.Nome, funcionario.Idade, funcionario.Sexo, funcionario.Salario);

            Console.WriteLine("O funcionário {0} estará de férias no período de 16/09/2020 à 14/09/2020", funcionario.Nome);
            funcionario.calculaFerias();

            Console.WriteLine("O funcionário {0} receberá R$ {1} Reais referente ao seu décimo terceiro", funcionario.Nome, funcionario.calculaDecimoTerceiro(12));
        }
    }
}
```

Os 4 pilares da Programação Orientada a Objetos

Para que uma linguagem possa ser enquadrada no paradigma de orientação a objetos, ela deve atender a **quatro tópicos** bastante importantes:



Os 4 pilares da Programação Orientada a Objetos

Encapsulamento

O *encapsulamento* é uma das principais técnicas que define a programação orientada a objetos.

Se trata de um dos elementos que adicionam segurança à aplicação em uma programação orientada a objetos pelo fato de esconder as propriedades, criando uma espécie de caixa preta.

Essa atitude evita o acesso direto a propriedade do objeto, adicionando uma outra camada de segurança à aplicação.

CEUB
EDUCAÇÃO SUPERIOR

Os 4 pilares da Programação Orientada a Objetos

Encapsulamento

- **Para fazermos um paralelo com o que vemos no mundo real, temos o encapsulamento em outros elementos.**
- **Por exemplo, quando clicamos no botão ligar da televisão, não sabemos o que está acontecendo internamente.**
- **Podemos então dizer que os métodos que ligam a televisão estão encapsulados.**

CEUB
EDUCAÇÃO SUPERIOR

Encapsulamento em C#

- O encapsulamento em C# ocorre nas classes.
- Quando o programador cria uma classe, ele especifica o Código e os dados que irão formar essa classe.
- Estes elementos serão chamados de membros da classe.
- O comportamento e a interface de uma classe são definidos pelos métodos que operam nas instâncias de dados.
- O encapsulamento em C# é implementado através dos seus modificadores de acesso público, protegido, privado e implícito (protect, private e public).

Encapsulamento em C#

- Considerando que o objetivo de uma classe é encapsular a complexidade, existem mecanismos para ocultar a complexidade da implementação que está dentro da classe.

- Cada método ou variável em uma classe pode ser definida como pública, privada ou protegida.

- A interface de uma classe possibilita que todos os usuários externos possam acessar livremente os dados da classe que os métodos públicos permitem.

- Já os métodos privados estabelecem que os dados somente podem ser acessados pelos métodos que são membros da classe.

Modificadores de acesso em C#

- **Public**: Com este modificador, o acesso é livre em qualquer lugar do programa.
- **Private**: Com este modificador, o acesso é permitido somente dentro da classe onde ele foi declarado. Por padrão, é a visibilidade definida para métodos e atributos em uma classe.
- **Protected**: Com este modificador, apenas a classe que contém o modificador e os tipos derivados dessa classe (HERANÇA) tem o acesso.
- **Internal**: Com este modificador, o acesso é limitado apenas ao assembly atual.
- **Protected Internal**: Com este modificador, o acesso é limitado ao assembly atual e aos tipos derivados da classe que contém o modificador.

CEUB
EDUCAÇÃO SUPERIOR

Modificadores de acesso em C#

- **Modificador de acesso private**

  - **Este modificador determina que o membro privado só pode ser acessado por métodos de dentro da própria classe.**

  - **O modificador de acesso private é o mais restritivo e que deve ser empregado sempre que possível.**

MODIFICADORES DE ACESSO

Imagine uma classe ContaBancaria que não tem um modificador de acesso;

Nesta classe, por mais que modifiquemos o atributo saldo através do método sacar, é possível atribuir qualquer valor ao atributo saldo.

```
class ContaBancaria {

    public int Numero { get; private set; }
    public string Titular { get; set; }
    public double Saldo { get; private set; }
    public ContaBancaria(int numero, string titular) {
        Numero = numero;
        Titular = titular;
    }
    public ContaBancaria(int numero, string titular, double saldo) : this(numero, titular) {
        Saldo = saldo;
    }
```

```
    public void Deposito(double quantia) {
        Saldo += quantia;
    }
    public void Saque(double quantia) {
        Saldo -= quantia + 5.0;
    }
    public override string ToString() {
        return "Conta "
            + Numero
            + ", Titular: "
            + Titular
            + ", Saldo: $ "
            + Saldo.ToString("F2", CultureInfo.InvariantCulture);
    }
}
```

MÉTODOS GETTERS E SETTERS

Properties

As propriedades também são responsáveis pela encapsulamento dos atributos. São definições de métodos encapsulados, porém expondo uma sintaxe similar à de atributos e não de métodos

Uma propriedade é um membro que oferece um mecanismo flexível para ler, gravar ou calcular o valor de um campo particular.

As propriedades podem ser usadas como se fossem atributos públicos, mas na verdade elas são métodos especiais chamados "acessadores".

Isso permite que os dados sejam acessados facilmente e ainda ajuda a promover a segurança e a flexibilidade dos métodos.

MÉTODOS GETTERS E SETTERS

Properties Exemplo

```
public class Pessoa
    {
        //definição dos atributos
        private string _nome;
        private int _idade;
        //definição das propriedades
        public string Nome
        {
            get { return _nome; }
            set { _nome = value; }
        }
        public int Idade
        {
            get { return _idade; }
            set{
                if (value > 120 || value < 0)
                    _idade = 0;
                else
                    _idade = value;
            }
        }
```

```
    //definição dos métodos
    public string VerificarIdade()
    {
        string msg;
        if (Idade > 18)
            msg = "Já pode Dirigir";
        else
            msg = "Não pode Dirigir";
        return msg;
    }
}
```

Auto Properties

Auto Properties (ou propriedades automáticas) em C# são uma sintaxe abreviada para declarar uma propriedade com um campo de suporte implícito.

Com auto properties, você não precisa criar explicitamente um campo de classe separado para armazenar o valor da propriedade.

Neste caso, o compilador cria um campo de classe anônimo automaticamente, que é usado para armazenar o valor da propriedade.

Auto Properties

Em C#, é possível declarar e inicializar um campo de classe diretamente na declaração da propriedade, usando a sintaxe "propriedade auto-implementada".

Essa sintaxe abreviada permite que você defina uma propriedade com um campo de suporte implícito, sem precisar criar explicitamente um campo de classe separado.

```
public class Pessoa {
    public string Nome { get; set; }
}
```

OBS: Em Java e Python, ainda é necessário declarar explicitamente os campos de classe antes de usá-los em propriedades. A propriedade em si é apenas uma abstração de um ou mais métodos que leem ou gravam o campo de classe.

Auto Properties

É uma forma simplificada de se declarar propriedades que não necessitam lógicas particulares para as operações get e set. Para gerar, selecione os atributos e com botão direito selecione opções rápidas e refatorações, encapsular campo, conforme descrito abaixo.

```
public class Pessoa
{
    //definição dos atributos
    private string _nome;
    private int _idade;

    0 referências
    public string Nome { get => _nome; set
    1 referência
```

Ações Rápidas e Refatorações... Ctrl+.
Renomear... Ctrl+R, Ctrl+R
Remover e Classificar Usos Ctrl+R, Ctrl+G
Exibir Código F7

```
12  private int _idade;
```

Usar a propriedade auto
Gerar construtor "Pessoa(int)"
Gerar Equals(object)
Gerar Equals e GetHashCode
Encapsular campo: "_idade" (e usar propriedade)
Encapsular campo: "_idade" (mas ainda usar o campo)
Suprimir ou Configurar os problemas

```
...
public string Nome { get => _nome; set => _nome = value; }
public int Idade { get => _idade; set => _idade = value; }
public int Idade { get => Idade1; set => Idade1 = value; }
public int Idade1 { get => _idade; set => _idade = value; }
...
```

Visualizar alterações

```
21  if (Idade > 18)
22      msg = "Já pode Dirigir";
```

```
public class Pessoa
{
    //definição dos atributos
    private string _nome;
    private int _idade;

    public string Nome { get => _nome; set => _nome = value; }
    public int Idade { get => _idade; set => _idade = value; }

    //definição dos métodos
    public string VerificarIdade()
    {
        string msg;
        if (Idade > 18)
            msg = "Já pode Dirigir";
        else
            msg = "Não pode Dirigir";
        return msg;
    }
}
```

EXERCÍCIO

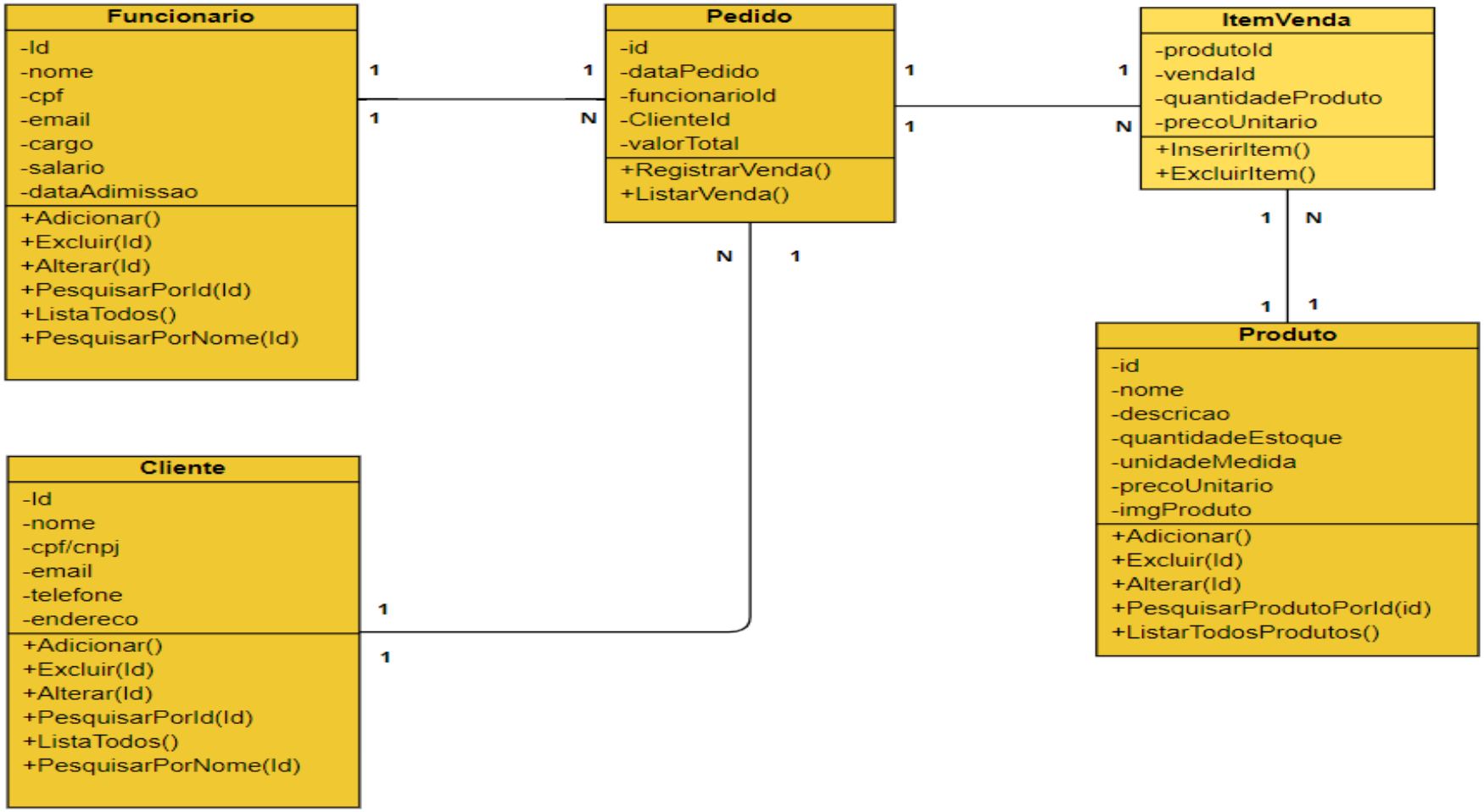
1 - Crie um atributo numero na classe ContaCorrente. Defina o atributo numero como private. Crie seus métodos get e set.

2- Crie um classe Funcionário que tenha os seguintes métodos privados : nome, dt_nascimento, salario, n_filhos, área_atuação. Crie seus métodos get e set.

3-Crie um classe teste funcionário que chame os elementos getters e setters da classe funcionário.

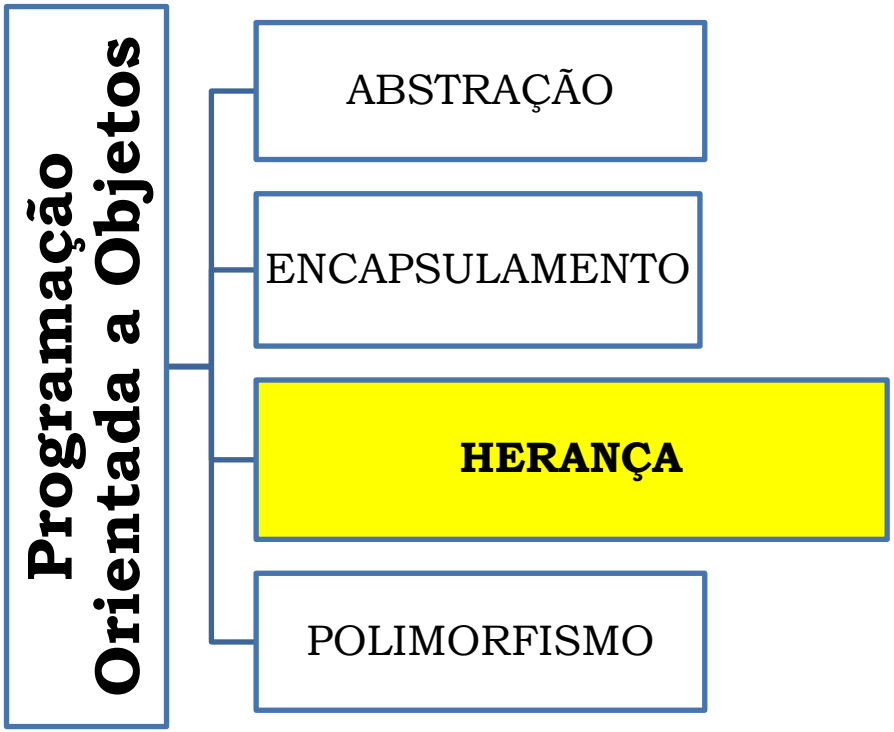
2 - EXERCÍCIO

Crie um projeto e implemente as suas classes usando os princípios da abstração e encapsulamento, de acordo com o diagrama abaixo



Os 4 pilares da Programação Orientada a Objetos

Para que uma linguagem possa ser enquadrada no paradigma de orientação a objetos, ela deve atender a **quatro tópicos** bastante importantes:



Os 4 pilares da Programação Orientada a Objetos

Herança

O reuso de código é uma das grandes vantagens da programação orientada a objetos. Muito disso se dá por uma questão que é conhecida como *herança*.

Essa característica otimiza a produção da aplicação em tempo e linhas de código.

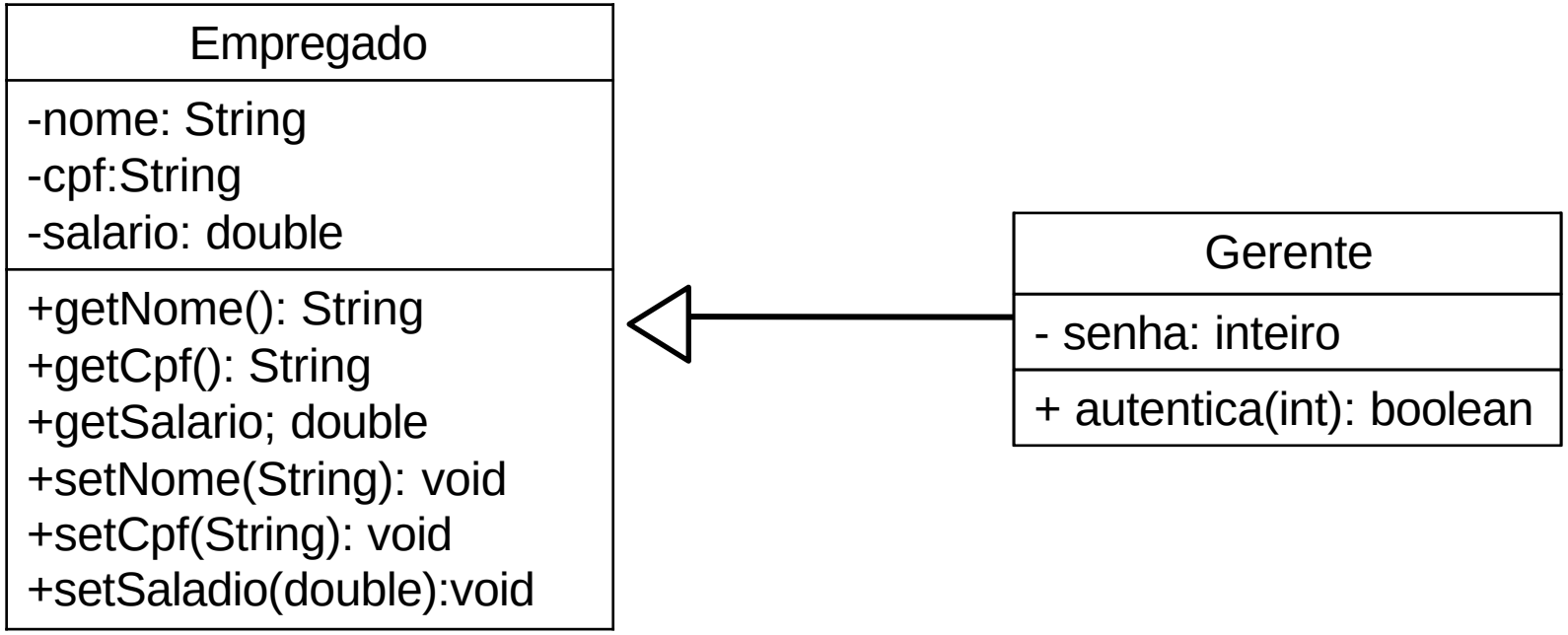
A reutilização de códigos nas linguagens orientadas a objetos é uma característica que otimiza o desenvolvimento de um aplicativo tanto em economia de tempo, quanto em número de linhas de código.

Herança - Relacionamento entre objetos

- Objetos não existem isolados
  - São formados por outros objetos
  - Objetos usam outros objetos
  - Um programa OO possui vários objetos que interagem entre si
  - Modelagem define quais objetos usamos em um programa

HERANÇA

Exemplo de notação UML



REPETIÇÃO DE CÓDIGO

Exemplo a classe Empregado, que representa o Empregado de um banco:

```
0 referências
class Empregado
{
    private string nome;
    private string cpf;
    private double salario;

    0 referências
    public string Nome { get => nome; set => nome = value; }
    0 referências
    public string Cpf { get => cpf; set => cpf = value; }
    0 referências
    public double Salario { get => salario; set => salario = value; }
}
```

REPETIÇÃO DE CÓDIGO

Classe Gerente:

```
0 referências
class Gerente
{
    private string nome;
    private string cpf;
    private double salario;

    0 referências
    public string Nome { get => nome; set => nome = value; }
    0 referências
    public string Cpf { get => cpf; set => cpf = value; }
    0 referências
    public double Salario { get => salario; set => salario = value; }
}
```

HERANÇA

Todo momento que criarmos um objeto do tipo Gerente, este objeto possuirá também os atributos definidos na classe Empregado, pois agora um Gerente é um Empregado!

Herança Exemplo

Exemplo
Vamos analisar a classe Pessoa e Motorista.

Se partirmos do pressuposto que um motorista é uma pessoa, podemos dizer que a Classe Motorista tem acesso aos membros da classe "pai" Pessoa.

Logo para que uma classe herde de outra ela deve usar a palavra reservada *:* *que* informa que a classe é filha de.

Já palavra reservada *base* chama o construtor da classe pai dentro do próprio construtor de classe

CONSTRUTOR

É da responsabilidade da subclasse inicializar os atributos definidos na sua classe, assim como os atributos que herda das suas superclasses.

O construtor da subclasse pode delegar a inicialização dos atributos herdados para a superclasse, chamando, implícita ou explicitamente, o construtor da superclasse.

Um construtor da subclasse pode fazer uma chamada explícita de um construtor da superclasse através do **base()**

CONSTRUTOR

Se o construtor da superclasse tiver N parâmetros, estes devem ser passados na chamada explícita:
base(param1,...,paramN).

Pode se chamar outro construtor da classe usando o this(parametros).

Se nenhum construtor da superclasse é chamado, ou se nenhum construtor da classe é chamado, como primeira instrução do construtor, o construtor sem argumentos da superclasse é implicitamente chamado antes de qualquer instrução no construtor.

Se a superclasse não tiver um construtor sem argumentos, é necessário chamar explicitamente um construtor da superclasse.

CEUB
EDUCAÇÃO SUPERIOR

CONSTRUTOR

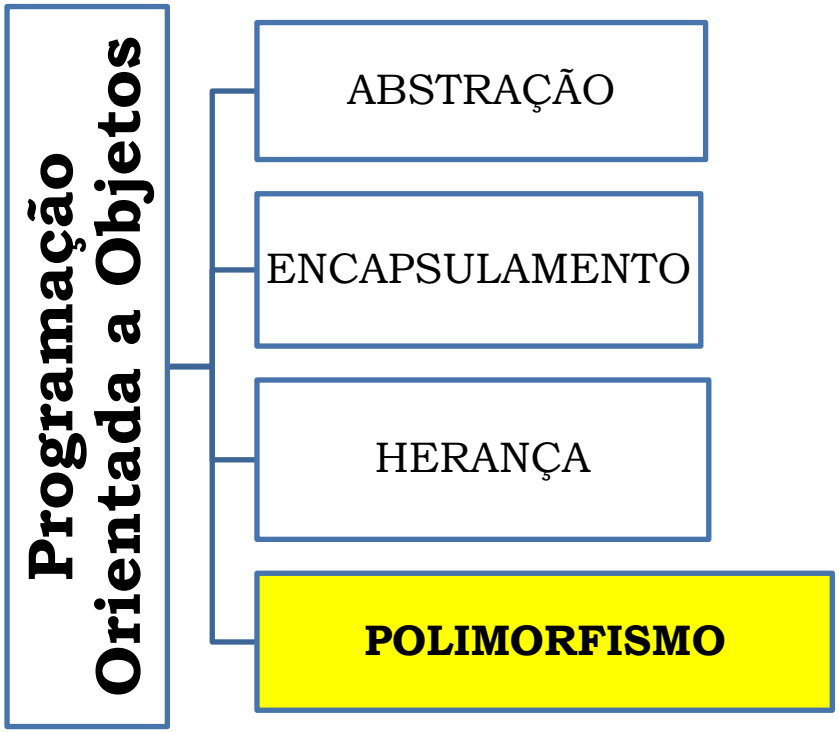
Pode se chamar outro construtor da classe usando o this(parametros).

Se nenhum construtor da superclasse é chamado, ou se nenhum construtor da classe é chamado, como primeira instrução do construtor, o construtor sem argumentos da superclasse é implicitamente chamado antes de qualquer instrução no construtor.

Se a superclasse não tiver um construtor sem argumentos, é necessário chamar explicitamente um construtor da superclasse.

Os 4 pilares da Programação Orientada a Objetos

Para que uma linguagem possa ser enquadrada no paradigma de orientação a objetos, ela deve atender a **quatro tópicos** bastante importantes:



Polimorfismo

O quarto e último pilar da POO é conhecido como polimorfismo. Trata-se da capacidade de alterar a forma original conforme a necessidade do momento.

A palavra polimorfismo vem do grego e significa muitas formas (poli: muitas, morphos: formas);

Em suma, o polimorfismo consiste na alteração de todo o funcionamento interno de um método herdado de um outro objeto dentro da aplicação.

Polimorfismo

No Polimorfismo temos dois tipos:

- **Polimorfismo Estático ou Sobrecarga**
- **Polimorfismo Dinâmico ou Sobreposição**

O Polimorfismo Estático se dá quando temos a mesma operação implementada várias vezes na mesma classe. A escolha de qual operação será chamada depende da assinatura dos métodos sobrecarregados.

Polimorfismo

O **Polimorfismo Dinâmico** acontece na herança, quando a subclasse sobrepõe o método original. Agora o método escolhido se dá em tempo de execução e não mais em tempo de compilação.

A escolha de qual método será chamado depende do tipo do objeto que recebe a mensagem.

Sobrecarga (overloading)

- **Sobrecarga é utilizada quando você escreve métodos com o mesmo nome mas com assinaturas diferentes.**

- **Uma boa prática é usar a sobrecarga, somente, nos métodos que possuam a mesma funcionalidade.**

- **A sobrecarga pode ser feita igualmente nos métodos construtores.**

EXEMPLO

```
public class ExemploSobrecarga {

   public int somar(int x, int y){
       return (x+y);
   }

   public int somar(int x, int y, int z){
       return (x+y+z);
   }
}
```

Sobrecarga (overloading)

Exemplo

Criar uma classe Calculadora com a seguinte estrutura

```
public class Calculadora {
    public int Soma(int numero1, int numero2) {
        return numero1 + numero2;
    }

    public double Soma(double numero1, double numero2) {
        return numero1 + numero2;
    }

    public int Soma(int numero1, int numero2, int numero3) {
        return numero1 + numero2 + numero3;
    }

    public float Soma(float numero1, float numero2) {
        return numero1 + numero2;
    }
}
```

Sobrecarga (overloading)

DIFERENÇAS this e Bases

- Usados quando for necessário referenciar explicitamente a instância (**this**) ou a superclasse (**Base**).

Em C#, this e base são palavras-chave que permitem acessar membros da classe atual e da classe base (classe pai), respectivamente.

A palavra-chave this é usada para se referir a um membro da classe atual. Por exemplo, se você tiver uma propriedade com o mesmo nome de um parâmetro em um método, pode usar this para se referir à propriedade em vez do parâmetro. Outro exemplo é quando você precisa passar a própria instância da classe como parâmetro para outro método, usando this você pode fazer isso de forma mais fácil.

Já a palavra-chave base é usada para se referir a um membro da classe base. Por exemplo, se a classe atual estender outra classe (classe base) e você quiser chamar um construtor da classe base, você pode usar base para fazer isso.

Sobrecarga (overloading)

DIFERENÇAS this e Bases

Em C#, this e base são palavras-chave que permitem acessar membros da classe atual e da classe base (classe pai), respectivamente.

A palavra-chave this é usada para se referir a um membro da classe atual. Por exemplo, se você tiver uma propriedade com o mesmo nome de um parâmetro em um método, pode usar this para se referir à propriedade em vez do parâmetro.

Outro exemplo é quando você precisa passar a própria instância da classe como parâmetro para outro método, usando this você pode fazer isso de forma mais fácil.

Sobrecarga (overloading)

DIFERENÇAS this e Bases

Já a palavra-chave base é usada para se referir a um membro da classe base.

Por exemplo, se a classe atual estender outra classe (classe base) e você quiser chamar um construtor da classe base, você pode usar base para fazer isso.

Sobrecarga (overloading)

DIFERENÇAS this e Bases

Exemplo

```
public class Pessoa {
    public string Nome { get; set; }

    public Pessoa(string nome) {
        this.Nome = nome;
    }
}

public class Aluno : Pessoa {
    public string Matricula { get; set; }

    public Aluno(string nome, string matricula) : base(nome) {
        this.Matricula = matricula;
    }
}
```

Nesse exemplo, a classe Aluno estende a classe Pessoa.
O construtor da classe Pessoa recebe o nome da pessoa como parâmetro, que é atribuído à propriedade Nome.

Na classe Aluno, o construtor recebe o nome e a matrícula do aluno como parâmetros. Para passar o nome para o construtor da classe base, a palavra-chave base é usada, e para atribuir a matrícula à propriedade Matricula da classe atual, a palavra-chave this é usada.

Sobrescrita (Override)

Exemplo Override

```
class Funcionario
{
     private string nome;
     private double salario;
     public string Nome { get => nome; set => nome = value; }
     public double Salario { get => salario; set => salario = value; }
     public virtual double getSalario(int horas)
     {
         salario = horas * 100;
         return salario;
     }
}
```

```
class Estagiario : Funcionario
{
    public override double getSalario(int horas)
    {
       return horas*50;
    }
}
```

```
class Diretor : Funcionario
{
   public override double getSalario(int horas)
   {
       return horas*200;
   }
}
```

Sobrescrita (Override)

Exemplo Override

```
namespace ExemploPolimorfismos
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Diretor diretor = new Diretor();
            Estagiario estagiario = new Estagiario();
            Console.WriteLine("A bonificação do Estagiário será de R$ {0} ", estagiario.getSalario(100));
            Console.WriteLine("A bonificação do Diretor será de R$ {0} ", diretor.getSalario(100));
        }
    }
}
```

Sobrescrita (Override)

Sobrescrita de método

Exemplo2.

- Vamos incluir a classe Funcionário um novo método bonificacao
- Esse método representa uma bonificação que todos os funcionários recebem no fim do ano e é referente a 10% do valor do salário. Porém, o gerente recebe uma bonificação de 15%.

- Como ficaria o código da nossa classe Funcionario?

Classes Abstratas

- **Ao se criar uma classe para ser estendida, é muito comum não se ter idéia de como codificar os seus métodos, isto é, somente as suas subclasses saberão implementá-los.**

- **Uma classe deste tipo não pode ser instanciada pois sua funcionalidade está incompleta. Tal classe é dita abstrata.**

- **OBS**
- **Ao se criar uma classe Abstrata, que tenha métodos abstratos, esses métodos deverão ser criados nas subclasses.**

Classes Abstratas

- Podem existir casos em que a classe se comporta como um tipo, logo, supomos que os objetos desse tipo não serão instanciados.
- Nesses casos, chamamos a classe de classe Abstrata.
- O único objetivo dessa classe é servir de superclasse para outras classes.
- As classes que herdam de classes abstratas são conhecidas como classes concretas.
- Temos uma característica exclusiva desse tipo de classe, os métodos abstratos.

Classes Abstratas

- No exemplo em questão, não faz sentido criar objetos do tipo Funcionário, mas sim objetos do tipo Diretor, Professor ou Administrador.

- Nesta caso, especificando Funcionario como classe abstrata, economiza-se código e ganha-se o poliformismo para a criação de métodos genéricos que servirão a diversos objetos.

- C# suporta o conceito de classes abstratas. Pode-se declarar uma classe abstrata usando o modificador abstract.

- Métodos também podem ser declarados abstratos com o modificador abstract. As suas implementações serão feitas nas subclasses.

- As classes abstratas podem ter métodos concretos, campos de dados e construtores. Os objetos das suas subclasses poderão fazer uso deles.

Classes Abstratas

- **Pode acontecer que ao escrever um método para uma classe você não saiba como ele vai ser implementado. Neste caso, a implementação será feita pela classe que herdar o método (a classe filha).**

- **Pode acontecer também que você saiba que um determinado método será sobreposto com certeza na classe filha; então, por que definir sua implementação se ela não será usada ?**

Classes Abstratas

- **Nestes casos você apenas define a assinatura do método e deixa a definição por conta da classe que irá herdar a classe pai.**
- **Estas classes são então chamadas classes abstratas, o método que você não implementou é chamado de método abstrato.**
- **As classes abstratas não podem ser instanciadas através da palavra chave New.**

Classes Abstratas

- Uma classe abstrata é uma classe base genérica
  - Contém métodos abstratos que devem ser implementados nas classes que derivam dela

- Um método abstrato não apresenta implementação na classe base

```
public abstract class Pessoa {
    public abstract void Cadastrar();
    public abstract string Nome { get; set; }
    public abstract int Id { get; }
    public virtual void Viajar() { /* Ação */ }
}
```

- Pode conter membros não-abstratos

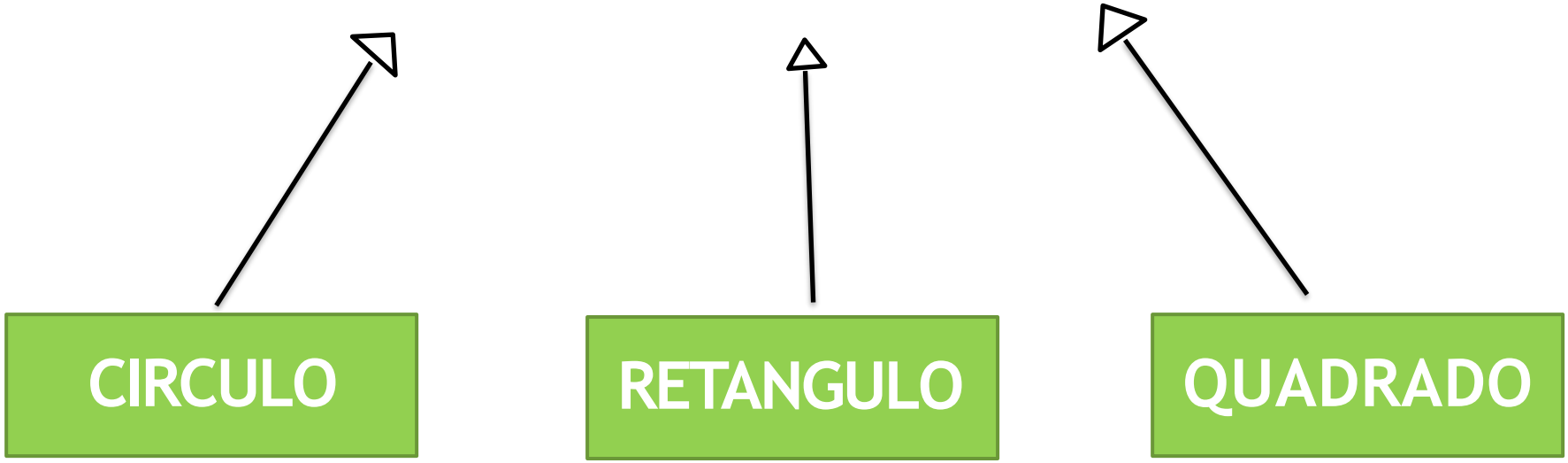
Classes Abstratas

- Derivando a classe abstrata e implementando os membros abstratos

```
public class Diretor : Pessoa
{
    public override void Cadastrar()
    { /* Ações */ }
    public override string Nome
    {
        get { /* Implementação get */ }
        set { /* Implementação set */ }
    }
    public override int Id
    {
        get { /* Implementação get */ }
    }
}
```

Classes Abstratas
Ex 2

```
public abstract class Forma {

    public abstract void desenhar();

    public abstract void informacoes();

    public void teste(){
        System.out.println("Testando!!");
    }
}
```



Classes Abstratas
Ex 3

```
public abstract class Personagem
{
    public string Nome { get; set; }
    public int Vida { get; set; }
    public int Nivel { get; set; }

    public abstract void UsarHabilidade();
}
```

```
public class Mago : Personagem
{
    public override void UsarHabilidade()
    {
        Console.WriteLine("O mago lançou um feitiço!");
    }
}
```


```
public class Mago : Personagem
{
    public override void UsarHabilidade()
    {
        Console.WriteLine("O mago lançou um feitiço!");
    }
}
```

Interface

Interfaces

- **Uma interface é parecida com uma classe abstrata, a diferença é que uma classe abstrata pode possuir métodos que não estejam implementados e pode possuir métodos que estejam implementados.**

- **Uma interface possui somente métodos que não estão implementados e que devem ser implementados pela classe que usar a interface.**

Interface

Na programação orientada a objetos, às vezes é útil definir o que uma classe deve fazer, mas não como ela o fará.

Já vimos um exemplo disso: o método abstrato.

Um método abstrato define a assinatura de um método, mas não fornece implementação

Uma subclasse deve fornecer sua própria implementação de cada método abstrato definido por sua superclasse. Portanto, um método abstrato especifica a interface do método, mas não a implementação.

Interface

Embora as classes e métodos abstratos sejam uteis, podemos levar esse conceito um passo adiante.

Em C#, podemos separar totalmente a interface de uma classe de sua implementação usando a palavra-chave interface.

Interface tem objetivo criar um "contrato" onde a Classe que a implementa deve obrigatoriamente obedecer.

Interface

- É um recurso utilizado para definir os métodos e propriedades de um determinado grupo de classes.

- Funcionam como um contrato, e todas as classes que participam do contrato deverão ter aqueles métodos e propriedades.

- O funcionamento do método em cada classe pode ser diferente, contanto que o método exista da maneira definida

Interface

- Dentro das interfaces existem apenas as assinaturas dos métodos e propriedades
- Cabe à classe realizar a implementação das assinaturas, dando comportamento concreto aos métodos
- Mais de uma classe pode implementar a mesma interface
- Uma classe pode implementar mais de uma interface

Interfaces

- **Como o C# não suporta herança múltipla as interfaces permitem que uma classe estenda múltiplas interfaces contornando o problema.**

- **Uma interface no C# não pode conter atributos, somente pode ter métodos, propriedades e eventos. Todos os membros de uma interface são públicos e não podem usar um modificador de acesso.**

Interfaces

- **A classe que implementa a interface deve possuir a definição de todos métodos existentes na interface. Esta definição deve possuir o mesmo nome e a mesma assinatura, retorno e parâmetros, do método na interface.**

- **O nome da classe e o nome da interface são separados por dois pontos(:).**

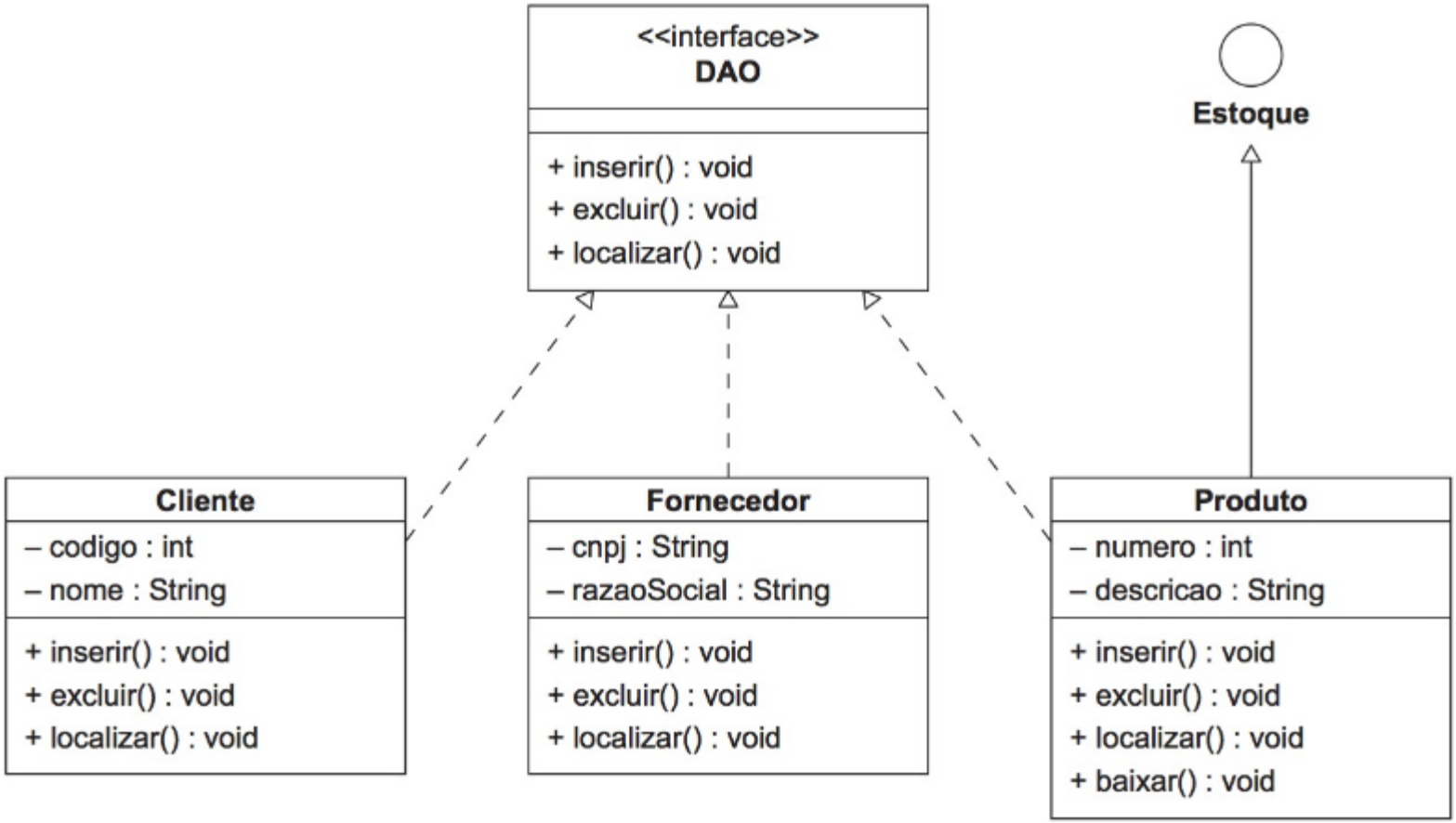
Interfaces

- **Uma interface define a mesma funcionalidade e comportamento à classes não relacionadas diretamente**
- **Declarando a interface**

```
public interface IProduto
{
    bool EPerecivel { get; }
    Fornecedor RecuperarFornecedor();
    void RegistrarVenda(Cliente cliente);
}
```

Exercício

- **Implemente o diagrama abaixo**



Referências:

https://owasp.org/www-community/attacks/

https://www.dio.me/articles/a-evolucao-da-arquitetura-de-software-backend

https://4future.com.br/index.php/2023/12/04/compreendendo-os-principios-solid/

https://medium.com/beelabacademy/princ%C3%ADpios-de-s-o-l-i-d-em-c-guia-pr%C3%A1tico-cbb1e6584284

PIRES, E. SOLID: Teoria e Prática.
ANICHE, M. Orientação a Objetos e SOLID para ninjas. São Paulo – Casa do Código

Obrigado